

```
df = pd.read_csv(HHS_Unaccompanied_Alien_Children_Program)
```

```
df.columns = ['Date', 'Apprehended_CBP', 'CBP_Custody', 'Transferred_Out', 'In_HHS_Care', 'Discharged_HHS']
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')
df = df.reset_index(drop=True)
```

```
# Convert 'In_HHS_Care' to numeric after removing commas
df['In_HHS_Care'] = df['In_HHS_Care'].astype(str).str.replace(',', '', regex=False)
df['In_HHS_Care'] = pd.to_numeric(df['In_HHS_Care'], errors='coerce')
```

```
print("Dataset Shape:", df.shape)
print(df.dtypes)
print(df.head())
print(df.isnull().sum())
```

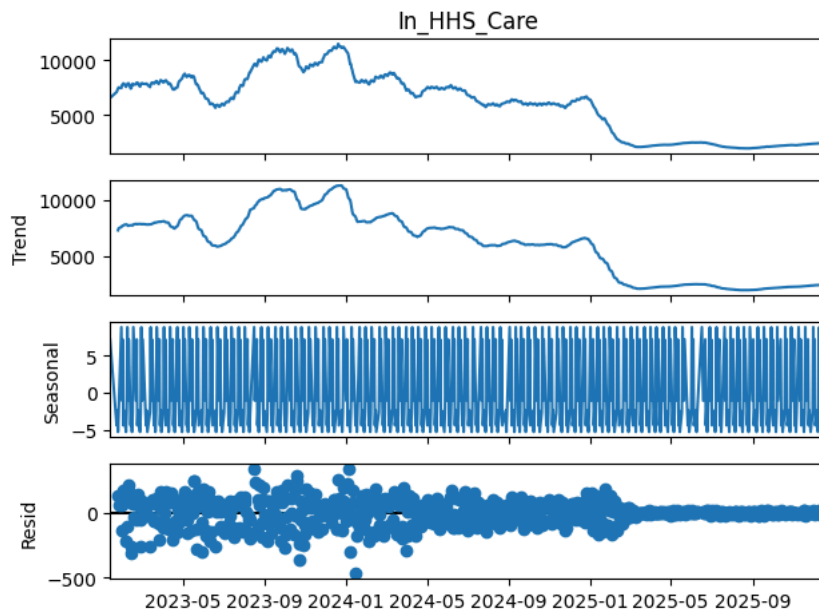
```
Dataset Shape: (1170, 6)
Date                datetime64[ns]
Apprehended_CBP      float64
CBP_Custody           float64
Transferred_Out       float64
In_HHS_Care           float64
Discharged_HHS        float64
dtype: object
   Date    Apprehended_CBP  CBP_Custody  Transferred_Out  In_HHS_Care  \
0 2023-01-12             33.0         53.0             34.0       6566.0
1 2023-01-22             32.0         49.0             39.0       7122.0
2 2023-01-23             32.0         50.0             39.0       7280.0
3 2023-01-24             47.0         42.0             47.0       7433.0
4 2023-01-25             20.0         22.0             41.0       7538.0

   Discharged_HHS
0             436.0
1             227.0
2             181.0
3             175.0
4             180.0
Date                450
Apprehended_CBP      450
CBP_Custody           450
Transferred_Out       450
In_HHS_Care           450
Discharged_HHS        450
dtype: int64
```

```
# --- HANDLE MISSING DATA ---
df.interpolate(method='linear', inplace=True)
```

```
# --- TIME-SERIES PREPARATION ---
df.set_index('Date', inplace=True)
daily_series = df['In_HHS_Care'] # main target variable
```

```
# Decompose the time series
from statsmodels.tsa.seasonal import seasonal_decompose
result = seasonal_decompose(daily_series, model='additive', period=7)
result.plot()
plt.show()
```



```
# --- FEATURE ENGINEERING ---
df['Lag_1'] = df['In_HHS_Care'].shift(1)
df['Lag_7'] = df['In_HHS_Care'].shift(7)
df['Lag_14'] = df['In_HHS_Care'].shift(14)
df['Rolling_Mean_7'] = df['In_HHS_Care'].rolling(window=7).mean()
df['Rolling_Mean_14'] = df['In_HHS_Care'].rolling(window=14).mean()
df['Day_of_Week'] = df.index.dayofweek
df['Month'] = df.index.month
df.fillna(method='bfill', inplace=True)
```

```
/tmp/ipykernel_579/131523156.py:9: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version
df.fillna(method='bfill', inplace=True)
```

```
# --- DEFINE TRAIN / TEST SPLIT ---
split_date = '2025-10-01'
train = df.loc[df.index <= split_date]
test = df.loc[df.index > split_date]

X_train = train.drop('In_HHS_Care', axis=1)
y_train = train['In_HHS_Care']
X_test = test.drop('In_HHS_Care', axis=1)
y_test = test['In_HHS_Care']
```

```
# MODELING SECTION
```

```
# --- 1. ARIMA MODEL ---
arima_model = ARIMA(y_train, order=(5,1,0))
arima_fit = arima_model.fit()
arima_forecast = arima_fit.forecast(steps=len(y_test))

# --- 2. RANDOM FOREST REGRESSOR ---
rf = RandomForestRegressor(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

# --- 3. GRADIENT BOOSTING REGRESSOR ---
gbr = GradientBoostingRegressor(n_estimators=200, learning_rate=0.05, random_state=42)
gbr.fit(X_train, y_train)
gbr_pred = gbr.predict(X_test)
```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available
return get_prediction_index(

```

```
# --- EVALUATION FUNCTION ---
def evaluate_model(y_true, y_pred, model_name):
```

```

mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
print(f"{model_name} -> MAE: {mae:.2f}, RMSE: {rmse:.2f}")

```

```

evaluate_model(y_test, arima_forecast, 'ARIMA')
evaluate_model(y_test, rf_pred, 'Random Forest')
evaluate_model(y_test, gbr_pred, 'Gradient Boosting')

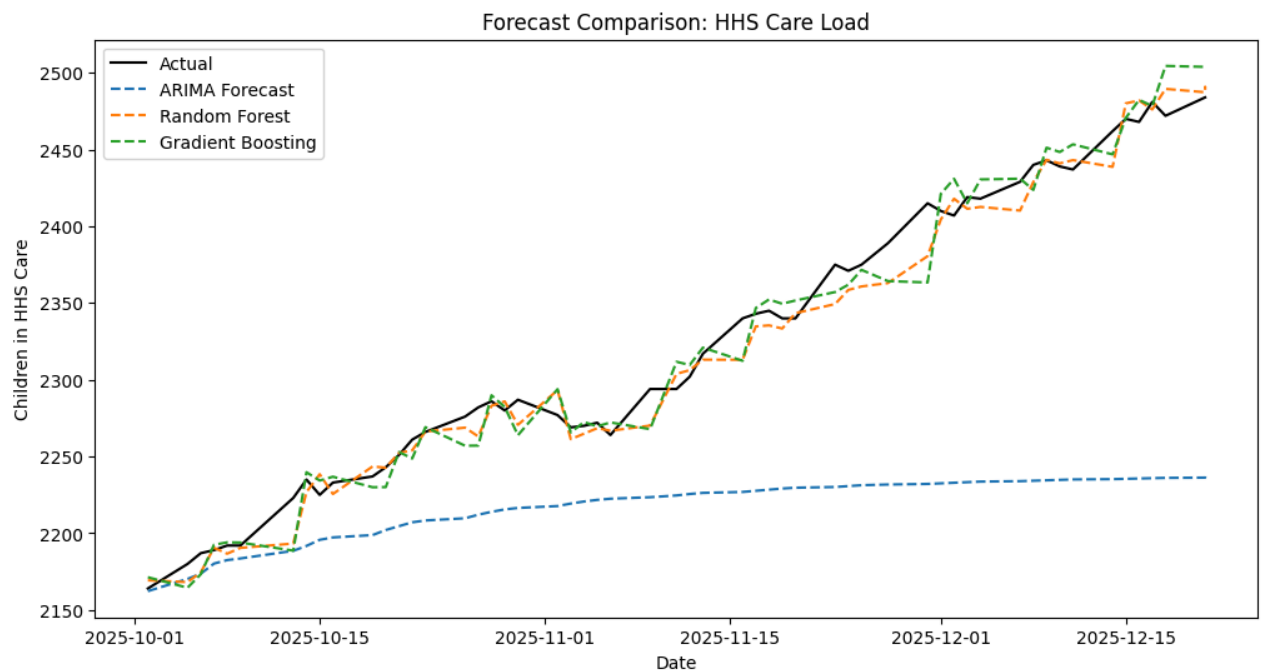
```

ARIMA -> MAE: 230.50, RMSE: 235.68
Random Forest -> MAE: 6.71, RMSE: 7.32
Gradient Boosting -> MAE: 19.04, RMSE: 19.47

```

# --- VISUALIZATION ---
plt.figure(figsize=(12,6))
plt.plot(y_test.index, y_test.values, label='Actual', color='black')
plt.plot(y_test.index, arima_forecast, label='ARIMA Forecast', linestyle='--')
plt.plot(y_test.index, rf_pred, label='Random Forest', linestyle='--')
plt.plot(y_test.index, gbr_pred, label='Gradient Boosting', linestyle='--')
plt.title('Forecast Comparison: HHS Care Load')
plt.xlabel('Date')
plt.ylabel('Children in HHS Care')
plt.legend()
plt.show()

```



```

# --- FEATURE IMPORTANCE (Random Forest) ---
importances = pd.Series(rf.feature_importances_, index=X_train.columns)
importances.nlargest(10).plot(kind='barh', figsize=(8,5), title='Feature Importances (Random Forest)')

```

<Axes: title={'center': 'Feature Importances (Random Forest)'}>

